

Set 19: Python Files

Skill 19.1: Read a file
Skill 19.2: Read lines in a file
Skill 19.3: Write a file
Skill 19.4: Append to a file
Skill 19.5: Read CSV files
Skill 19.6: Read different types of CSV files
Skill 19.7: Write to a CSV file
Skill 19.8: Read a JSON file
Skill 19.9: Write to a JSON file

Skill 19.1: Read a file

Skill 19.1 Concepts

Computers use file systems to store and retrieve data. Each file is an individual container of related information. If you've ever saved a document, downloaded a song, or even sent an email you've created a file on some computer somewhere.

Here we are going to learn how to read and write different kinds of files using code. Let's say we had a file called **document.txt** with these contents: "Wowsers!"

We could read this file like this,

Code	document.txt
<pre>my_doc = open('document.txt') text = my_doc.read() print(text)</pre>	Wowsers!
Output	
Wowsers!	

While the code above gets the job done in that it prints the contents of the file, it does not close the file, which can lead to *file locking*. File locking is an issue that arises when a file is opened but not closed. If a file is left open, other programs are blocked from accessing it.

The code below fixes the issue and is the preferred way to read files,

Code	document.txt
<pre>with open('document.txt') as my_doc: contents = my_doc.read() print(contents)</pre>	Wowsers!
Output	
Wowsers!	

Below is explanation for what is happening in the code segment above,

1. `open('document.txt')`

- Opens the file named **document.txt**.
- By default, it opens the file in **read mode** ('r').
- Python creates a *file object* that lets you work with the file's contents.

2. `with ... as my_doc:`

- This is a **context manager**.
- It automatically:
 - Opens the file at the start
 - **Closes the file when the block ends**, even if an error occurs
- `my_doc` is the variable that represents the open file.

3. `my_doc.read()`

- Reads **the entire contents of the file** as a single string.
- That string (including newlines `\n`) is stored in `contents`.

Skill 19.1 Exercise 1

Skill 19.2: Read lines in a file

Skill 19.2 Concepts

When we read a file, we might want to grab the whole document in a single string like `read()` would return. But what if we wanted to store each line in a variable? We can use the `readlines()` method to read a text file line by line instead of reading the whole thing at once.

The `readlines()` method returns a list of lines that are in the text,

Code	sonnet.txt
<pre>with open('sonnet.txt') as sonnet: print(sonnet.readlines())</pre>	To one who has been long in city pent, 'Tis very sweet to look into the fair And open face of heaven,-to breathe a prayer Full in the smile of the blue firmament.
Output	
['To one who has been long in city pent,\n', "'Tis very sweet to look into the fair\n", 'And open face of heaven,-to breathe a prayer\n', 'Full in the smile of the blue firmament.\n']	

We can therefore use a loop to print out the contents of each line,

Code	sonnet.txt
<pre>with open('sonnet.txt') as sonnet: for line in sonnet.readlines(): print(line)</pre>	To one who has been long in city pent, 'Tis very sweet to look into the fair And open face of heaven,-to breathe a prayer Full in the smile of the blue firmament.
Output	
To one who has been long in city pent, 'Tis very sweet to look into the fair And open face of heaven,-to breathe a prayer Full in the smile of the blue firmament.	

Sometimes you don't want to iterate through a whole file. For that, there's a different file method, `readline()`, which will only read a single line at a time. If the entire document is read line by line in this way subsequent calls to `readline()` will not throw an error but will start returning an empty string.

Code	millay_sonnet.txt
<pre>with open('millay_sonnet.txt') as sonnet_doc: first_line = sonnet_doc.readline() second_line = sonnet_doc.readline() print(second_line)</pre>	I shall forget you presently, my dear, So make the most of this, your little day, Your little month, your little half a year, Ere I forget, or die, or move away,
Output	
So make the most of this, your little day,	

[Skill 19.2 Exercises 1 and 2](#)

Skill 19.3: Write a file

Skill 19.3 Concepts

Reading a file is all well and good, but what if we want to create a file of our own? With Python we can do that with the same `open()` function we used previously. Creating a file can be done by passing an additional argument, `'w'`, to the `open()` function.

```
with open('generated_file.txt', 'w') as gen_file:
    gen_file.write("What an incredible file!")
```

In the example above, we pass the argument `'w'` to `open()` to indicate to open the file in write-mode. The default argument is `'r'` and passing `'r'` to `open()` opens the file in read-mode as we've been doing.

The code above creates a new file in the same folder as `script.py` and gives it the text, *What an incredible file!* It's important to note that if there is already a file called `generated_file.txt` it will completely overwrite that file, erasing whatever its contents were before.

[Skill 19.3 Exercise 1](#)

Skill 19.4: Append to a file

Skill 19.4 Concepts

Files can be appended to by using the `'a'` argument in the `open()` function.

Code
<pre>with open('bad_bands.txt', 'a') as gen_file: gen_file.write("Bad 80s Bands: ") with open('bad_bands.txt', 'a') as gen_file: gen_file.write("\n Twisted Sister")</pre>
bad_bands.txt
Bad 80s bands: Twisted Sister

In the first code block above we open a file object in the temporary variable *gen_file*. Because we are using the 'a' argument, if the *bad_bands.txt* exists we will go into append-mode, otherwise the file, *bad_bands.txt* will be created and *Bad 80s Bands:* will be appended.

In the next code block we repeat the above. Because *bad_bands.txt* exists, *\nTwisted Sister* is appended to the file. The newline character *\n* ensures that the new text is added on a new line.

Skill 19.4 Exercise 1

Skill 19.5: Read CSV files

Skill 19.5 Concepts

Text files aren't the only thing that Python can read, but they're the only thing that doesn't require an additional parsing library to understand. CSV files are an example of a text file that impose a structure to their data. CSV stands for *Comma-Separated Values* and CSV files are usually the way that data from spreadsheet software (like Microsoft Excel or Google Sheets) is exported into a portable format. A spreadsheet that looks like the following

Name	Username	Email
Roger Smith	rsmith	wigginsryan@yahoo.com
Michelle Beck	mlbeck	hcosta@hotmail.com
Ashley Barker	a_bark_x	a_bark_x@turner.com
Lynn Gonzales	goodmanjames	lynniegonz@hotmail.com
Jennifer Chase	chasej	jchase@ramirez.com
Charles Hoover	choover	choover89@yahoo.com
Adrian Evans	adevans	adevans98@yahoo.com
Susan Walter	susan82	swilliams@yahoo.com
Stephanie King	stephanieking	sking@morris-tyler.com
Erika Miller	jessica32	ejmiller79@yahoo.com

In a CSV file that same exact data would be rendered like this,

users.txt
Name,Username,Email
Roger Smith,rsmith,wigginsryan@yahoo.com
Michelle Beck,mlbeck,hcosta@hotmail.com
Ashley Barker,a_bark_x,a_bark_x@turner.com
Lynn Gonzales,goodmanjames,lynniegonz@hotmail.com
Jennifer Chase,chasej,jchase@ramirez.com
Charles Hoover,choover,choover89@yahoo.com
Adrian Evans,adevans,adevans98@yahoo.com
Susan Walter,susan82,swilliams@yahoo.com
Stephanie King,stephanieking,sking@morris-tyler.com
Erika Miller,jessica32,ejmiller79@yahoo.com

Notice that the first row of the CSV file doesn't actually represent any data, just the labels of the data that's present in the rest of the file. The rest of the rows of the file are the same as the rows in the spreadsheet software, just instead of being separated into different cells they're separated by... commas, of course!

Even though we can read these lines as text without a problem,

```
with open('users.csv') as log_csv_file:
    print(log_csv_file.read())
```

There are ways to access the data in a format better suited for programming purposes. In Python we can convert that data into a dictionary using the CSV library's *DictReader* object.

Code

```
import csv
with open('users.csv', newline='') as users_csv:
    user_reader = csv.DictReader(users_csv)
    for row in user_reader:
        print(row)
```

Output

```
{'Name': 'Roger Smith', 'Username': 'rsmith', 'Email': 'wigginsryan@yahoo.com'}
{'Name': 'Michelle Beck', 'Username': 'mlbeck', 'Email': 'hcosta@hotmail.com'}
{'Name': 'Ashley Barker', 'Username': 'a_bark_x', 'Email': 'a_bark_x@turner.com'}
{'Name': 'Lynn Gonzales', 'Username': 'goodmanjames', 'Email': 'lynniegonz@hotmail.com'}
{'Name': 'Jennifer Chase', 'Username': 'chasej', 'Email': 'jchase@ramirez.com'}
{'Name': 'Charles Hoover', 'Username': 'choover', 'Email': 'choover89@yahoo.com'}
{'Name': 'Adrian Evans', 'Username': 'adevans', 'Email': 'adevans98@yahoo.com'}
{'Name': 'Susan Walter', 'Username': 'susan82', 'Email': 'swilliams@yahoo.com'}
{'Name': 'Stephanie King', 'Username': 'stephanieking', 'Email': 'sking@morris-tyler.com'}
{'Name': 'Erika Miller', 'Username': 'jessica32', 'Email': 'ejmiller79@yahoo.com'}
```

In the above code we first import the *csv* library, which gives us the tools to parse our CSV file. Then we open the *users.csv* file with the temporary variable *users_csv*.

We pass the additional keyword argument, *newline=""*, to the *open()* function so that we don't accidentally mistake a line break in one of our data fields as a new row in our CSV (read more about this in [the Python documentation](#)).

After opening our new CSV file, we use *csv.DictReader(users_csv)* to convert each line of our CSV file to a Python dictionary which we can use a loop to access.

Notice, the keys of the dictionary are, by default, the entries in the first line of the CSV file.

Skill 19.5 Exercise 1

Skill 19.6: Read different types of CSV files

Skill 19.6 Concepts

So far, we've been acting like all CSV files are *Comma-Separated Value* files. It's true that CSV stands for that, but it's also true that other ways of separating values are valid CSV files.

People used to call Tab-Separated Values files TSV files, but as other separators grew in popularity everyone realized that creating a new `[a-z]sv` file format for every value-separating character used is not sustainable.

So, we call all files with a list of different values a CSV file and then use different *delimiters* (like a comma or tab) to indicate where the different values start and stop.

Let's say we had an address book. Since addresses usually use commas in them, we'll need to use a different delimiter for our information. Since none of our data has semicolons (;) in them, we can use those.

addresses.csv
Name;Address;Telephone Donna Smith;126 Orr Corner Suite 857\nEast Michael, LA 54411;906-918-6560 Aaron Osborn;6965 Miller Station Suite 485\nNorth Michelle, KS 64364;815.039.3661x42816 Jennifer Barnett;8749 Alicia Vista Apt. 288\nLake Victoriaberg, TN 51094;397-796-4842x451 Joshua Bryan;20116 Stephanie Stravenue\nWhitneytown, IA 87358;(380)074-6173 Andrea Jones;558 Melissa Keys Apt. 588\nNorth Teresahaven, WA 63411;+57(8)7795396386 Victor Williams;725 Gloria Views Suite 628\nEast Scott, IN 38095;768.708.3411x954

Notice the `\n` character, this is the escape sequence for a new line. The possibility of a new line escaped by a `\n` character in our data is why we pass the `newline=""` argument to the `open()` function.

Also notice that many of these addresses have commas in them! This is okay, we'll still be able to read the file because each address is separated by a semicolon.

Code
<pre>with open('addresses.csv', newline = "") as user_addresses: address_dict = csv.DictReader(user_addresses, delimiter = ";") for row in address_dict: print(row)</pre>
Output
<pre>{'Name': 'Donna Smith', 'Address': '126 Orr Corner Suite 857\nEast Michael, LA 54411', 'Telephone': '906-918-6560'} {'Name': 'Aaron Osborn', 'Address': '6965 Miller Station Suite 485\nNorth Michelle, KS 64364', 'Telephone': '815.039.3661x42816'} {'Name': 'Jennifer Barnett', 'Address': '8749 Alicia Vista Apt. 288\nLake Victoriaberg, TN 51094', 'Telephone': '397-796-4842x451'} {'Name': 'Joshua Bryan', 'Address': '20116 Stephanie Stravenue\nWhitneytown, IA 87358', 'Telephone': '(380)074-6173'} {'Name': 'Andrea Jones', 'Address': '558 Melissa Keys Apt. 588\nNorth Teresahaven, WA 63411', 'Telephone': '+57(8)7795396386'} {'Name': 'Victor Williams', 'Address': '725 Gloria Views Suite 628\nEast Scott, IN 38095', 'Telephone': '768.708.3411x954'}</pre>

Notice in the example above, when we call `csv.DictReader` we pass in the `delimiter` parameter to indicate string used to delineate separate fields in the CSV. We then iterate through the CSV and print out each of the addresses.

Skill 19.6 Exercise 1

Skill 19.7: Write to a CSV file

Skill 19.7 Concepts

Sometimes we need to be able to programmatically create CSV files that save output and data that someone could load into their spreadsheet software. Let's say we have a big list of data that we want to save into a CSV file. We could do the following,

Code

```
big_list = [{'name': 'Fredrick Stein', 'userid': 6712359021, 'is_admin': False},
{'name': 'Wiltmore Denis', 'userid': 2525942, 'is_admin': False}, {'name': 'Greely Plonk', 'userid': 15890235, 'is_admin': False}, {'name': 'Dendris Stulo', 'userid': 572189563, 'is_admin': True}]

import csv

with open('output.csv', 'w') as output_csv:
    fields = ['name', 'userid', 'is_admin']
    output_writer = csv.DictWriter(output_csv, fieldnames=fields)
    output_writer.writeheader()
    for item in big_list:
        output_writer.writerow(item)
```

output.csv

```
name,userid,is_admin

Fredrick Stein,6712359021,False

Wiltmore Denis,2525942,False

Greely Plonk,15890235,False

Dendris Stulo,572189563,True
```

Below breaks down what the code above does,

1. **with open('output.csv', 'w') as output_csv:**
 - Opens (or creates) output.csv in **write mode**.
 - If the file already exists, it will be **overwritten**.
 - The file will be automatically closed at the end of the block.
2. **fields = ['name', 'userid', 'is_admin']**
 - Defines the **column headers** for the CSV file.

3. **csv.DictWriter(output_csv, fieldnames=fields)**
 - Creates a CSV writer that expects each row to be a **dictionary**.
 - Dictionary keys should match the field names.
4. **output_writer.writeheader()**
 - Writes the header row: *name,userid,is_admin*
5. **for item in big_list:**
 - Loops through *big_list*, which should be a **list of dictionaries**.
6. **output_writer.writerow(item)**
 - Writes one row per dictionary.
 - Example dictionary: `{'name': 'Alice', 'userid': 42, 'is_admin': False}`

Skill 19.7 Exercise 1

Skill 19.8: Read a JSON file

Skill 19.8 Concepts

CSV isn't the only file format that Python has a built-in library for. We can also use Python's file tools to read and write JSON. JSON, an abbreviation of JavaScript Object Notation, is a file format inspired by the programming language JavaScript. The name, like CSV is a bit of a misnomer — some JSON is not valid JavaScript (and plenty of JavaScript is not valid JSON).

JSON's format is endearingly similar to Python dictionary syntax, and so JSON files might be easy to read from a Python developer standpoint. Nonetheless, Python comes with a `json` package that will help us parse JSON files into actual Python dictionaries. Suppose we have a JSON file like the following,

<code>purchase.json</code>
<pre>{ "user": "ellen_greg", "action": "purchase", "item_id": "14781239" }</pre>

The following could load the file to a Python dictionary,

Code
<pre>import json with open('purchase.json') as purchase_json: purchase_data = json.load(purchase_json) print(purchase_data['user'])</pre>
Output
ellen_greg

Below is an explanation for the code above,

1. **import json**
 - Imports Python's built-in JSON library so Python can work with JSON data.
2. **with open('purchase.json') as purchase_json:**
 - Opens the file purchase.json in read mode.
 - The file is automatically closed afterward.
3. **json.load(purchase_json)**
 - Reads the JSON file and converts it into a Python data structure.
4. **purchase_data = ...**
 - Stores the converted data in purchase_data.
5. **print(purchase_data['user'])**
 - Accesses the value associated with the key "user" in the dictionary.
 - Prints that value.

Skill 19.8 Exercise 1

Skill 19.9: Write to a JSON file

Skill 19.9 Concepts

Naturally we can use the json library to translate Python objects to JSON as well. This is especially useful in instances where you're using a Python library to serve web pages, you would also be able to serve JSON. Let's say we had a Python dictionary we wanted to save as a JSON file,

```
turn_to_json = {  
    'eventId': 674189,  
    'dateTime': '2015-02-12T09:23:17.511Z',  
    'chocolate': 'Semi-sweet Dark',  
    'isTomatoAFruit': True  
}
```

We'd be able to create a JSON file with that information by doing the following,

Code

```
with open('output.json', 'w') as json_file:  
    json.dump(turn_to_json, json_file)
```

output.json

```
{"eventId": 674189, "dateTime": "2015-02-12T09:23:17.511Z", "chocolate": "Semi-sweet  
Dark", "isTomatoAFruit": true}
```

In the example above, we first import the *json* module. We then opened up a write-mode file under the variable *json_file*. Finally, we used the *json.dump()* method to write to the file.

Note that `json.dump()` takes two arguments: first the data object, then the file object you want to save.

Skill 19.9 Exercise 1